PONDICHERRY UNIVERSITY

(A Central University)



SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

Master of Computer Science (Data Analytics)

NAME	**:**	**MURSHIDA P**
REG.NO.	**:**	**26MSCSCPY0010**
SEMESTER	**:**	**I**
SUBJECT	**:**	**CSDA606 : DESIGN AND ANALYSIS OF ALGORITHMS LAB**

PONDICHERRY UNIVERSITY

(A Central University)

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

Master of Computer Science (Data Analytics)

This is to certify that this is a Bonafide record of practical work done by **MURSHIDA P** having Reg.No.**26MSCSCPY0010** M.Sc. Computer Science (Data Analytics) in I Semester from the month July to November in 2026-2027.

FACULTY IN-CHARGE

SUBMITTED FOR THE PRACTICAL EXAM HELD ON:_____/____/_____

EXAMINER

INDEX

Ex. No.	Date	Title	Page	Signature
1.	28-07-2026	Binary Search	1	
2.	28-07-2026	Find Maximum and Minimum	3	
3.	18-08-2026	Strassen's Matrix Multiplication	4	
4.	18-08-2026	Job Sequencing Using Greedy Method	6	
5.	18-08-2026	Prim's Algorithm	8	
6.	01-09-2026	Traveling Salesman Problem	10	
7.	15-09-2026	Sum of Subsets Using Backtracking	12	
8.	08-09-2026	Travelling Salesman Problem Using Branch and Bound	14	

SOURCE CODE :

```
#include <stdio.h>
int main()
{
   int a[100], n, key, low, high, mid, i;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   printf("Enter elements in ascending order:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);
   printf("Enter element to search: ");
   scanf("%d", &key);
   low = 0;
   high = n - 1;
   while(low <= high)
   {
      mid = (low + high) / 2;
      if(a[mid] == key)
      {
         printf("Element found at position %d", mid + 1);
         return 0;
      }
      else if(key < a[mid])
         high = mid - 1;
      else
         low = mid + 1;
   }
   printf("Element not found");
   return 0;
}
```

Ex. No: 01

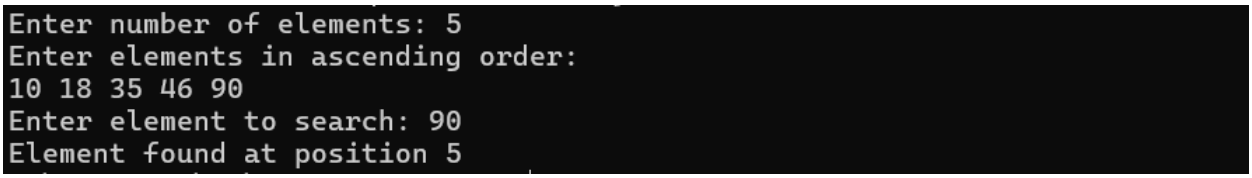
Date: 28-07-2026

BINARY SEARCH

AIM :

ALGORITHM :

OUTPUT :


Enter number of elements: 5
Enter elements in ascending order:
10 18 35 46 90
Enter element to search: 90
Element found at position 5

RESULT :

SOURCE CODE :

```
#include <stdio.h>
int main()
{
   int a[100], n, i, min, max;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   printf("Enter the elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);
   min = max = a[0];
   for(i = 1; i < n; i++)
   {
      if(a[i] < min)
         min = a[i];
      if(a[i] > max)
         max = a[i];
   }
   printf("Minimum element = %d\n", min);
   printf("Maximum element = %d\n", max);
   return 0;
}
```

OUTPUT :

```
Enter number of elements: 7
Enter the elements:
30 15 40 118 2 27 50
Minimum element = 2
Maximum element = 118
```

Ex. No: 02

Date: 28-07-2026

MAXIMUM AND MINIMUM

AIM :

ALGORITHM :

RESULT :

SOURCE CODE :

```
#include <stdio.h>
int main()
{
   int a[4][4], b[4][4], c[4][4];
   int m1[2][2], m2[2][2], m3[2][2], m4[2][2];
   int m5[2][2], m6[2][2], m7[2][2];
   int p[2][2], q[2][2], r[2][2], s[2][2];
   int i, j, k;
   printf("Enter elements of matrix A:\n");
   for(i = 0; i < 4; i++)
      for(j = 0; j < 4; j++)
         scanf("%d", &a[i][j]);
   printf("Enter elements of matrix B:\n");
   for(i = 0; i < 4; i++)
      for(j = 0; j < 4; j++)
         scanf("%d", &b[i][j]);

   for(i = 0; i < 2; i++)
   {
      for(j = 0; j < 2; j++)
      {
         m1[i][j] = (a[i][j] + a[i+2][j+2]) * (b[i][j] + b[i+2][j+2]);
         m2[i][j] = (a[i+2][j] + a[i+2][j+2]) * b[i][j];
         m3[i][j] = a[i][j] * (b[i][j+2] - b[i+2][j+2]);
         m4[i][j] = a[i+2][j+2] * (b[i+2][j] - b[i][j]);
         m5[i][j] = (a[i][j] + a[i][j+2]) * b[i+2][j+2];
         m6[i][j] = (a[i+2][j] - a[i][j]) * (b[i][j] + b[i][j+2]);
         m7[i][j] = (a[i][j+2] - a[i+2][j+2])* (b[i+2][j] + b[i+2][j+2]);
      }
   }
```

Ex. No: 03

Date: 18-08-2026

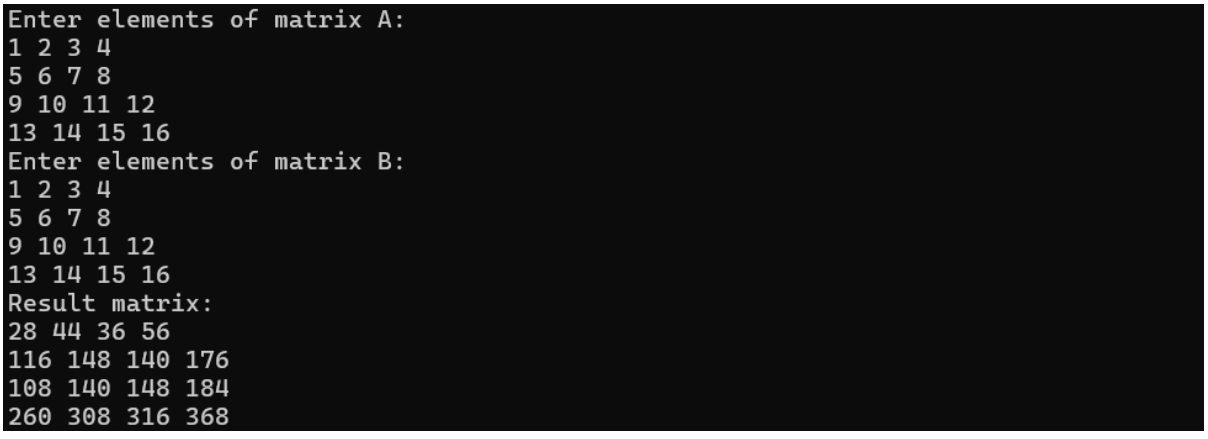
STRASSEN'S MATRIX MULTIPLICATION

AIM :

ALGORITHM :

```
for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 2; j++)
        {
            c[i][j] = m1[i][j] + m4[i][j] - m5[i][j] + m7[i][j];
            c[i][j+2] = m3[i][j] + m5[i][j];
            c[i+2][j] = m2[i][j] + m4[i][j];
            c[i+2][j+2] = m1[i][j] - m2[i][j] + m3[i][j] + m6[i][j];
        }
    }
    printf("Result matrix:\n");
    for(i = 0; i < 4; i++)
    {
        for(j = 0; j < 4; j++)
            printf("%d ", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT :



RESULT :

SOURCE CODE :

```
#include <stdio.h>
int main()
{
   int n, i, j, max = 0, total = 0, temp;
   int deadline[20], profit[20], job[20];
   int slot[20], maxDeadline = 0;
   printf("Enter number of jobs: ");
   scanf("%d", &n);
   printf("Enter job ID, deadline and profit:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d%d%d", &job[i], &deadline[i], &profit[i]);
      if(deadline[i] > maxDeadline)
         maxDeadline = deadline[i];
   }
   for(i = 0; i < n - 1; i++)
   {
      for(j = 0; j < n - i - 1; j++)
      {
         if(profit[j] < profit[j + 1])
         {
            temp = profit[j];
            profit[j] = profit[j + 1];
            profit[j + 1] = temp;
            temp = deadline[j];
            deadline[j] = deadline[j + 1];
            deadline[j + 1] = temp;
            temp = job[j];
            job[j] = job[j + 1];
            job[j + 1] = temp;
         }
      }
```

Ex. No: 04

Date: 18-08-2026

JOB SEQUENCING USING GREEDY METHOD

AIM :

ALGORITHM :

```
    }
    for(i = 0; i < maxDeadline; i++)
        slot[i] = -1;

    for(i = 0; i < n; i++)
    {
        for(j = deadline[i] - 1; j >= 0; j--)
        {
            if(slot[j] == -1)
            {
                slot[j] = job[i];
                total += profit[i];
                break;
            }
        }
    }
    printf("\nJob Schedule:\n");
    for(i = 0; i < maxDeadline; i++)
    {
        if(slot[i] != -1)
            printf("Slot %d: Job %d\n", i + 1, slot[i]);
    }
    printf("Maximum Profit = %d\n", total);
    return 0;
}
```

OUTPUT :

```
Enter number of jobs: 5
Enter job ID, deadline and profit:
1 2 100
2 1 19
3 2 27
4 1 25
5 3 15

Job Schedule:
Slot 1: Job 3
Slot 2: Job 1
Slot 3: Job 5
Maximum Profit = 142
```

RESULT :

SOURCE CODE :

```
#include <stdio.h>
int main()
{
   int a[20][20], visited[20] = {0};
   int n, i, j, edges = 0;
   int min, u = 0, v = 0, total = 0;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter adjacency matrix (0 for no edge):\n");
   for(i = 0; i < n; i++)
      for(j = 0; j < n; j++)
         scanf("%d", &a[i][j]);
   visited[0] = 1;
   printf("Edges in MST:\n");
   while(edges < n - 1)
   {
      min = 999999;
      for(i = 0; i < n; i++)
      {
         if(visited[i])
         {
            for(j = 0; j < n; j++)
            {
               if(!visited[j] && a[i][j] != 0 && a[i][j] < min)
               {
                  min = a[i][j];
                  u = i;
                  v = j;
               }
            }
         }
      }
```

Ex. No: 05

Date: 18-08-2026

PRIM'S ALGORITHM

AIM :

ALGORITHM :

```
 if(min == 999999)
      {
         printf("Graph is disconnected\n");
         return 0;
      }
      printf("%d - %d : %d\n", u, v, min);
      visited[v] = 1;
      total += min;
      edges++;
   }
   printf("Total weight = %d\n", total);
   return 0;
}
```

OUTPUT :

```
Enter number of vertices: 5
Enter adjacency matrix (0 for no edge):
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0
Edges in MST:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
Total weight = 16
```

RESULT :

SOURCE CODE :

```
#include <stdio.h>
int cost[10][10], path[10], bestPath[10];
int visited[10], n, minCost = 999999;
void tsp(int city, int count, int total)
{
   int i;
   if(count == n)
   {
      if(cost[city][0] != 0 &&
        total + cost[city][0] < minCost)
      {
         minCost = total + cost[city][0];
         for(i = 0; i < n; i++)
            bestPath[i] = path[i];
      }
      return;
   }
   for(i = 1; i < n; i++)
   {
      if(!visited[i] && cost[city][i] != 0)
      {
         visited[i] = 1;
         path[count] = i;
         tsp(i, count + 1, total + cost[city][i]);
         visited[i] = 0;
      }
   }
}
int main()
{
   int i, j;
   printf("Enter number of cities (maximum 10): ");
```

Ex. No: 06

Date: 01-09-2026

TRAVELLING SALESMAN PROBLEM

AIM :

ALGORITHM :

```
    scanf("%d", &n);
    if(n < 2 || n > 10)
    {
        printf("Invalid number of cities");
        return 0;
    }
    printf("Enter cost matrix (0 for no route):\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    path[0] = 0;
    visited[0] = 1;
    tsp(0, 1, 0);
    if(minCost == 999999)
        printf("No complete tour exists\n");
    else
    {
        printf("Minimum cost: %d\n", minCost);
        printf("Tour: ");
        for(i = 0; i < n; i++)
            printf("%d -> ", bestPath[i]);
        printf("0\n");
    }
    return 0;
}
```

OUTPUT :

```
Enter number of cities (maximum 10): 4
Enter cost matrix (0 for no route):
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Minimum cost: 80
Tour: 0 -> 1 -> 3 -> 2 -> 0
```

RESULT :

SOURCE CODE :

```
#include <stdio.h>
int a[20], x[20], n, target;
void sumOfSubsets(int i, int sum)
{
   int j;
   if(sum == target)
   {
      printf("{ ");
      for(j = 0; j < i; j++)
         if(x[j] == 1)
            printf("%d ", a[j]);
      printf("}\n");
      return;
   }
   if(i == n || sum > target)
      return;
   x[i] = 1;
   sumOfSubsets(i + 1, sum + a[i]);
   x[i] = 0;
   sumOfSubsets(i + 1, sum);
}

int main()
{
   int i;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   printf("Enter the elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);
   printf("Enter target sum: ");
```

Ex. No: 07

Date: 15-09-2026

SUM OF SUBSETS USING BACKTRACKING

AIM :

ALGORITHM :

```
    scanf("%d", &target);
    printf("Subsets with sum %d:\n", target);
    sumOfSubsets(0, 0);
    return 0;
}
```

OUTPUT :

```
Enter number of elements: 5
Enter the elements:
10 17 32 9 8
Enter target sum: 50
Subsets with sum 50:
{ 10 32 8 }
```

RESULT :

SOURCE CODE :

```
#include <stdio.h>
int n, cost[10][10], visited[10];
int path[10], bestPath[10];
int minCost = 999999;
void tsp(int city, int count, int total)
{
   int i;
   if(count == n)
   {
      if(cost[city][0] != 0)
      {
         total += cost[city][0];
         if(total < minCost)
         {
            minCost = total;
            for(i = 0; i < n; i++)       bestPath[i] = path[i];
         }
      }
      return;
   }
    for(i = 0; i < n; i++)
    {
      if(!visited[i] && cost[city][i] != 0)
       {
         if(total + cost[city][i] < minCost)
         {
            visited[i] = 1;
            path[count] = i;
            tsp(i, count + 1, total + cost[city][i]);
            visited[i] = 0;
         }
       }
```

Ex. No: 08

Date: 08-09-2026

TSP USING BRANCH AND BOUND

AIM :

ALGORITHM :

```
  }
}
int main()
{
    int i, j;
    printf("Enter number of cities: ");
    scanf("%d", &n);
    printf("Enter cost matrix (0 for no path):\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    visited[0] = 1;
    path[0] = 0;
    tsp(0, 1, 0);
    if(minCost == 999999)
        printf("No valid tour exists.\n");
    else
    {
        printf("Optimal tour: ");
        for(i = 0; i < n; i++)
            printf("%d -> ", bestPath[i]);
        printf("0\n");
        printf("Minimum cost = %d\n", minCost);
    }
    return 0;
}
```

OUTPUT :

```
Enter number of cities: 4
Enter cost matrix (0 for no path):
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Optimal tour: 0 -> 1 -> 3 -> 2 -> 0
Minimum cost = 80
```

RESULT :